# MpFA: Hardware-Efficient Train-Free QK4V8 FlashAttention Kernels on Blackwell GPUs

Chencheng Deng, Jianbin Fang, Dezun Dong*
College of Computer Science and Technology, National University of Defense Technology
Changsha, China
{chenchengdeng,j.fang,dong}@nudt.edu.cn

arXiv:2609.33135v2 [cs.DC] 10 Nov 2026

## Abstract

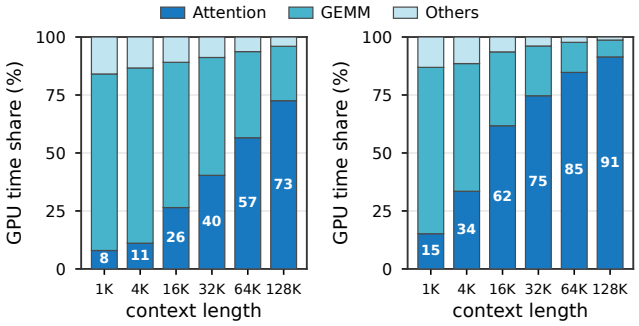
Long-context LLM inference pushes modern GPU serving stacks into an attention-bound regime, where both compute and memory are dominated by the softmax–GEMM pipeline. On NVIDIA Blackwell GPUs, FP4 Tensor Cores offer high matmul throughput, but we find that fully FP4 attention often fails to translate this throughput into end-to-end speedups due to non-matmul costs: online quantization after softmax, tensor/shared-memory data movement, and contention on the softmax path. We present MpFA, a training-free FlashAttention kernel optimized for Blackwell. Guided by hardware characterization, MpFA uses mixed precision: NVFP4 for $QK^\top$ and FP8 for $PV$ (QK4PV8). This preserves low-bit $QK^\top$ throughput while avoiding the conversion and scaling overheads of FP4 $PV$. To recover accuracy without further stressing the softmax pipeline, MpFA introduces rank-one smoothing compensation implemented as an additional Tensor Core MMA. MpFA further improves performance with a fine-grained asynchronous pipeline, tensor-memory reuse, and adaptive parallel partitioning across prefill and decode. On an NVIDIA SM100 GPU and across 16K–128K contexts, MpFA improves prefill throughput over state-of-the-art BF16/FP8 baselines and increases end-to-end output throughput by 2.81× over BF16 FA4 across Llama-3.1-8B and Qwen3-14B. Across five benchmark suites and two models, rank-one compensation recovers 62.5% of the accuracy loss with about 2.0% kernel overhead.

***Keywords:*** FlashAttention, Quantization, Blackwell GPU

## 1 Introduction

Applications such as retrieval-augmented generation, code understanding, and multi-turn agents increasingly rely on long contexts, making long-context inference an important LLM-serving workload [1, 12]. Meanwhile, AI infrastructure is adopting the Blackwell architecture. NVIDIA's data-center SM100 GPUs combine fifth-generation Tensor Cores, native low-bit computation, and tensor memory for storing operands and intermediate results.

LLM inference consists of prefill and decode. Prefill processes the entire input sequence, so its attention computation grows approximately quadratically with context length. Decode generates one token at a time and repeatedly accesses the growing KV cache, resulting in memory traffic that increases linearly with context length and low arithmetic intensity. We run Llama-3.1-8B with SGLang's standard BF16 FA4 backend on an SM100 GPU. As shown in Figure 1, attention accounts for 72.5% of device-side GPU time in prefill and 91.4% in decode at 128K tokens. Although FlashAttention-4 (FA4) [27] pipelines computation through tensor memory and fifth-generation Tensor Cores, the evaluated backend uses BF16 and does not exploit FP4 Tensor Cores for the two dominant matrix multiplications.




**(a)** prefill **(b)** decode

**Figure 1.** Device-side GPU time breakdown for Llama-3.1-8B served by SGLang on an SM100 GPU with the BF16 FA4 backend.

Quantization-aware training can adapt a model to 4-bit attention, but requires costly per-model retraining and is difficult to deploy at scale. Post-training methods avoid retraining; for example, SageAttention3 [30] applies NVFP4 attention to consumer Blackwell SM120 GPUs (RTX 5090), while HiFA4 [6] targets the Ascend HIF4 NPU. However, neither is designed to exploit SM100's tensor memory, fifth-generation Tensor Cores, and operand-delivery paths or to jointly support decode optimization, paged KV caches, and CUDA Graph capture. Efficient training-free FP4 attention therefore remains unavailable on data-center Blackwell GPUs.

Our hardware characterization and profiling reveal two challenges in building training-free FP4 FlashAttention for data-center Blackwell GPUs. First, when fifth-generation Tensor Cores execute block-scaled FP4 matrix multiply-add (MMA), matrix operands must be supplied from shared memory, whereas scale factors must come from tensor memory.

*Corresponding author.

Quantizing the softmax output $P$ to FP4 adds per-block amax computation, scale-factor generation, and data movement between tensor and shared memory. These operations load the softmax path, preventing low-bit matrix-multiplication speedups from translating into end-to-end performance. Second, existing smoothing compensation methods place the compensation term on the softmax path. This extra element-wise work further loads the already saturated path, where it competes with other operations and offsets the benefit of FP4 computation.

We present MpFA, a high-performance, training-free low-bit attention kernel for LLM inference on data-center Blackwell GPUs. We characterize three precision assignments: NVFP4 for both matrix multiplications; NVFP4 for $QK^\top$ with BF16 for $PV$; and NVFP4 for $QK^\top$ with FP8 for $PV$. Our evaluation shows that QK4PV8 provides the best balance between arithmetic throughput and non-matmul overhead on SM100, making it the hardware-efficient choice for FA4. FP8 $PV$ retains the benefits of low-precision Tensor Core execution while avoiding the additional precision-conversion requirements of an FP4 probability matrix. BF16 $PV$ avoids these conversion requirements as well, but leaves Tensor Core throughput on the table because FP8 Tensor Cores provide twice the throughput of BF16. MpFA therefore adopts QK4PV8 mixed precision and optimizes the FA4 pipeline for both inference stages: prefill uses a fine-grained asynchronous pipeline and tensor-memory column reuse to co-locate scale factors with double-buffered accumulators; decode restores parallelism through GQA packing, KV partitioning, and column-split softmax while avoiding exponential evaluations on padded rows.

To reduce smoothing overhead, MpFA introduces a rank-one Tensor Core MMA compensation method that moves the correction from the saturated softmax path to otherwise underutilized Tensor Cores. It stores $K$ in NVFP4 and $V$ in FP8, consuming the low-bit KV cache without dequantization and reducing KV bytes per token to 1/2.56 of BF16.

Experiments on SM100 show that, across 16K–128K, MpFA achieves average prefill speedups of 1.31–1.33× over BF16 FA4 and 1.15–1.17× over FP8 FlashInfer. Across 8K–128K, its decode kernel achieves a 2.08× average speedup over split-KV-enabled BF16 FA4. Across ten model–task pairs spanning five standard benchmarks, rank-one smoothing recovers 62.5% of the accuracy loss on average, with about 2.0% average kernel overhead. MpFA achieves 2.81× the end-to-end output throughput of BF16 FA4 across both models and 16K–128K, and by 1.06× over FP8 FlashInfer across both models and 32K–128K contexts.

This paper makes the following contributions:

1. We systematically analyze how attention kernels on data-center Blackwell GPUs become limited by non-matmul units, providing insight for optimizing other fused operators (§3).
2. We present MpFA, a high-performance QK4PV8 Flash-Attention kernel for LLM inference, with tensor memory reuse and stage-specific optimizations for prefill and decode (§5).
3. We introduce a rank-one Tensor Core smoothing algorithm that moves compensation from the saturated softmax path to MMA, recovering accuracy while maintaining hardware utilization (§4).

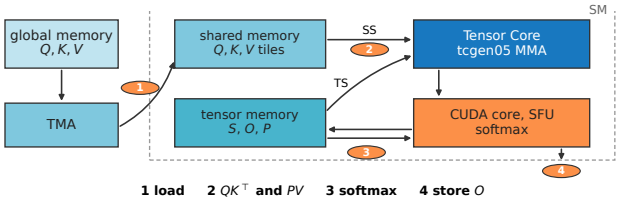
## 2 Background

### 2.1 Blackwell GPUs

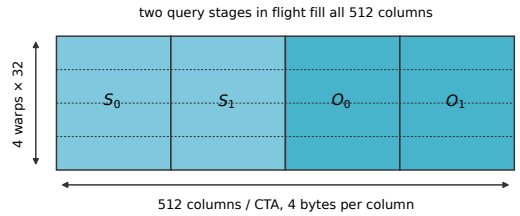
**Blackwell** is NVIDIA's GPU architecture for next-generation AI computing. This paper focuses on data-center SM100 rather than consumer SM120 (RTX 5090). Both provide Blackwell Tensor Cores and low-bit formats, but differ in compute capabilities and operand delivery. SM120 attention kernels are optimized for the consumer GPU's matrix multiply-add (MMA) path, whereas SM100 introduces on-chip tensor memory and `tcgen05.mma`. Consequently, SM120 implementations cannot fully exploit SM100, whose adoption requires attention kernels redesigned around its data path [13].

**From hardware to kernels.** SM100's Tensor Cores, tensor memory, and shared memory jointly determine the on-chip data path for attention. Figure 2 summarizes the execution path considered in this paper. In Figure 2(a), the asynchronous copy engine (TMA) first moves tiled $Q$, $K$, and $V$ from global memory to shared memory. Tensor Cores then use `tcgen05.mma` instructions to compute $QK^\top$ and $PV$ and write the intermediate results to tensor memory. CUDA cores and special function units execute online softmax and directly read from and write to the score, probability, and output accumulators in tensor memory. The final results are written back to global memory. The SS and TS labels in the figure denote two operand-delivery paths; as discussed in §3.1, these paths constrain the precision assignment. Unlike traditional execution models, which keep accumulators in registers, this design uses tensor memory as separate on-chip storage for results, allowing matrix multiplication and softmax to form an asynchronous pipeline with explicit role separation [27]. Figure 2(b) shows the capacity constraint: in our kernel configuration, each cooperative thread array (CTA) is allocated 128 rows × 512 columns, or 256 KB. Two attention score accumulators and two output accumulators, each with block size 128 and FP32 elements, consume all 512 columns.

**Block-scaled FP4 matrix multiply-add.** FP4 cannot represent the full dynamic range of activations by itself, so SM100 uses block-scaled FP4 MMA. NVFP4 associates every consecutive $G$=16 elements along the reduction dimension with one `e4m3` scale factor; by comparison, MXFP4 uses $G$=32 elements and one `e8m0` scale factor. For a block $x_g \in \mathbb{R}^G$,




**(a)** FlashAttention4 data path




**(b)** Tensor memory architecture

**Figure 2.** Data path and tensor memory organization.

quantization and hardware reconstruction can be written as

$$s_g = \frac{\text{amax}(x_g)}{\rho}, \qquad \hat{x}_g = \left\lfloor \frac{x_g}{s_g} \right\rceil \text{e2m1}, \qquad \tilde{x}_g = s_g \hat{x}_g, \quad (1)$$

where $\rho$=6 is the maximum magnitude of `e2m1`. In Equation 1, the Tensor Core performs the reconstruction $\tilde{x}_g = s_g \hat{x}_g$ internally during MMA. Block scaling limits quantization error locally, but introduces scale factors that must accompany the operands. We use NVFP4 because its 16-element blocks and scales better match attention activations than MXFP4 [30]. Different MMA precisions impose different operand-source and scale-factor requirements, making hardware details critical to both precision selection and performance bottlenecks.

### 2.2 FlashAttention and Quantization

Self-attention comprises two matrix multiplications and one softmax:

$$S = QK^\top/\sqrt{d}, \qquad P = \text{softmax}(S), \qquad O = PV \quad (2)$$

FlashAttention [3, 4] tiles $Q$, $K$, and $V$ along the sequence dimension and processes KV blocks with online softmax, keeping intermediates on chip. Let the score for the $t$-th KV block be $S^{(t)} = QK^{(t)\top}/\sqrt{d}$. FlashAttention maintains a row-wise maximum $m$, normalization factor $\ell$, and output accumulator $O$:

$$m^{(t)} = \max\left(m^{(t-1)}, \text{rowmax}(S^{(t)})\right) \quad (3)$$

$$P^{(t)} = e^{S^{(t)} - m^{(t)}} \quad (4)$$

$$\ell^{(t)} = e^{m^{(t-1)} - m^{(t)}} \ell^{(t-1)} + \text{rowsum}\left(P^{(t)}\right) \quad (5)$$

$$O^{(t)} = \text{diag}\left(e^{m^{(t-1)} - m^{(t)}}\right) O^{(t-1)} + P^{(t)} V^{(t)} \quad (6)$$

Row maximum, exponentiation, row sum, and output rescaling in Equations 3–6 form the non-matmul softmax path. FA3 [23] hides its latency through warp specialization, overlapping a tile's matmul with the preceding tile's softmax. FA4 [27] reorganizes this pipeline for Blackwell tensor memory and fifth-generation Tensor Cores (Figure 2(a)).

**Quantized attention.** Unlike static weights in linear-layer quantization, $Q$, $K$, $P$, and $V$ are runtime activations coupled by softmax, making quantized attention sensitive to outliers, normalization, and accumulation error. The SageAttention series [29–31] and HiFA4 [6] demonstrate that post-training quantization can accelerate attention without retraining. It can also consume low-bit KV caches directly [16], reducing memory use and avoiding dequantization.

## 3 Motivation

Low-bit attention can exploit SM100's FP4 Tensor Cores while compressing the KV cache, reducing the compute and memory costs of long-context inference. However, Table 1 shows that existing systems target consumer Blackwell GPUs or NPUs, without addressing SM100's tensor memory, operand-delivery paths, or serving-time decode.

### 3.1 Asymmetric Hardware Scaling

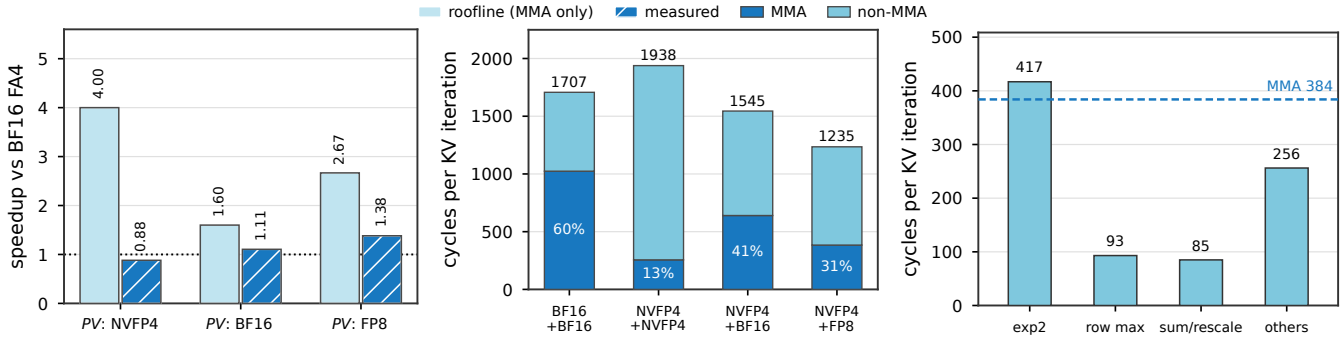
Blackwell's gains concentrate in Tensor Cores: SM100 delivers 2.25 PFLOPS in BF16, compared with 1 PFLOPS on SM90, and 9 PFLOPS in FP4 [17, 19]. Exponential throughput and shared-memory bandwidth have not scaled proportionally [13, 18, 27], so reducing matmul precision yields no comparable reduction in softmax cost. The SM100 operand-delivery paths further amplify this asymmetry. Block-scaled FP4 MMA requires its operands to reside in shared memory and its scale factors to reside in tensor memory, whereas FP8 MMA can source its first operand directly from tensor memory. Moreover, BF16 FA4 double buffers the queries, while its two FP32 score and output accumulators, each with a tile size of 128, already occupy all 512 tensor memory columns available to each CTA, leaving limited capacity for additional scale factors. Precision selection is therefore jointly constrained by compute throughput, data movement, and contention for tensor memory capacity.

We implement and measure four configurations on SM100: BF16 FA4, and three configurations that fix $QK^\top$ to NVFP4 while using NVFP4, BF16, or FP8 for $PV$. Based only on Tensor Core MMA cycles, NVFP4 $PV$ should be fastest, followed by FP8 and then BF16, with theoretical speedups of 4.00×, 2.67×, and 1.60×, respectively. However, Figure 3(a) shows the opposite measured ordering. At $L$=64K, NVFP4 + FP8 is 1.38× faster than BF16 FA4, NVFP4 + BF16 achieves 1.11×, while NVFP4 + NVFP4 reaches only 0.87× and is slower than BF16 FA4. The result demonstrates that precision selection on SM100 cannot be based on Tensor Core peak throughput alone.

Figure 3(b) decomposes the SM cycles per KV-block iteration and explains why the theoretical matrix-multiplication speedup does not translate into end-to-end operator speedup. In the BF16 configuration, the two matrix multiplications

**Table 1.** Positioning relative to the closest related systems.

| Work | Hardware | $QK^\top$ / $PV$ | Compensation | Partitioned decode | Workload |
|---|---|---|---|---|---|
| FA4 [27] | SM100 | BF16 / BF16 | — | Incompatible with graph capture | Vision/language |
| FlashInfer [26] | SM100 | FP8 / FP8 | — | Graph-compatible | Vision/language |
| SageAttn [31] | SM89 | INT8 / INT8 | Softmax | — | Vision/language |
| SageAttn2 [29] | SM90 | INT4 / FP8 | Softmax | — | Vision/language |
| SageAttn3 [30] | SM120 | NVFP4 / NVFP4 | Softmax | — | Vision |
| HiFA4 [6] | Ascend NPU | HIF4 / HIF4 | Softmax | — | Language |
| **MpFA (ours)** | **SM100** | **NVFP4 / FP8** | **MMA** | **Graph-compatible** | **Language** |




**(a)** $QK^\top$ fixed to NVFP4 **(b)** Performance gap **(c)** Performance breakdown

**Figure 3.** Four precision assignments for causal prefill at $L$=64K with GQA 32:8.

consume 1024 cycles, or 60% of total operator time. The BF16 FA4 implementation already approximates the exponential with a third-order polynomial on the CUDA cores to reduce the cost of the softmax exponential, and we adopt the same technique. NVFP4 + FP8 reduces this component to 384 cycles, a 63% reduction, but total time falls only from 1707 to 1235 cycles, or 28%. Softmax, reductions, and data movement do not shrink with matrix-multiplication precision. Once attention enters the 4-bit regime, the dominant constraint therefore shifts from matrix multiplication to the softmax path.

The reason NVFP4 + NVFP4 becomes slower is operand delivery rather than FP4 MMA throughput. If the probability matrix $P$, just generated in tensor memory by softmax, is used as an operand for block-scaled NVFP4 MMA, three additional costs arise. First, $P$ must be quantized by computing an absolute maximum over every 16 elements and generating scale factors; this work is placed on the already busy softmax path. Second, block-scaled MMA requires both operands from shared memory, so $P$ must leave tensor memory, be written to shared memory, and then be read by the matrix multiplication unit. Third, $P$'s scale factors need tensor memory capacity, competing with the score and output accumulators that already consume all 512 columns and requiring additional scale-factor transfers.

The FP8 $PV$ path avoids all three costs. Without block scaling, FP8 MMA can read its first operand directly from tensor memory, allowing the probability matrix to remain in place. FP8 Tensor Cores also provide twice the throughput of BF16. Thus, an efficient SM100 precision assignment must consider both data format and operand-delivery path: NVFP4($QK^\top$)+FP8($PV$) is the hardware-efficient point.

### 3.2 Inefficient Accuracy Compensation

Figure 3(c) further breaks down the execution time of NVFP4 + FP8. We obtain the softmax-stage costs through a compile-time ablation chain: we first rebuild the kernel with `exp2` disabled, then additionally disable the row-max and row-sum/rescale stages, and compute successive differences. This procedure makes the stage costs additive.

The softmax path consumes 595 cycles, or 48% of total operator time, with `exp2` alone accounting for 417 cycles, exceeding the 384 cycles of the two quantized matrix multiplications. An $M$=$N$=128 KV block contains 16384 score elements. At 16 elements per SM cycle, fully serialized exponential evaluation would require 1024 cycles. The measured 417 cycles are lower because CUDA core polynomial evaluation overlaps with exponential unit execution, while matrix multiplication overlaps with the softmax path. Thus, once attention enters the 4-bit regime, non-matmul computation,

particularly the softmax path, becomes the dominant bottleneck.

Low-bit quantization still requires accuracy compensation. Prior methods typically smooth outliers by subtracting the means of $Q$ and $K$ and adding a compensation factor back to the attention scores. The compensation must be added before masking and before updating the running maximum; otherwise it breaks online-softmax normalization. Existing approaches consequently place additional loads and element-wise additions on the softmax bottleneck. In our measurements, this placement costs 18.6–22.7% of operator time, consuming more than half of the advantage of the unsmoothed low-bit attention.

### 3.3 Insights from Profiling and Analysis

The analysis yields three principles for low-bit attention on data-center Blackwell GPUs.

**Principle 1: mixed precision.** NVFP4($QK^\top$)+FP8($PV$) accelerates $QK^\top$ computation without quantizing $P$ to NVFP4, avoiding the associated data-movement and scale management overheads along the softmax path.

**Principle 2: reduce non-matmul overhead.** Rank-one Tensor Core MMA compensation moves the cost from softmax to Tensor Cores, while a fine-grained pipeline improves utilization.

**Principle 3: specialize prefill and decode.** Prefill processes the complete query sequence and is primarily constrained by the softmax path. Decode has only a small number of valid query rows at each step and is primarily constrained by insufficient parallelism and padded rows introduced by MMA atom.

## 4 Quantization Algorithm

Following the first two principles in §3, MpFA uses NVFP4 for $QK^\top$, FP8 for $PV$, and sequence centering with BF16 rank-one Tensor Core MMA to compensate for channel-outlier-induced NVFP4 error in $Q$ and $K$.

### 4.1 QK4PV8 Quantization

MpFA quantizes $Q$ and $K$ to NVFP4 with one `e4m3` scale factor per 16 head-dimension elements, and quantizes $P$ and $V$ to FP8. The online softmax states $m$, $\ell$, and the $PV$ accumulator remain FP32, and the operator returns BF16. This assignment reduces $QK^\top$ computation and KV storage without the block-scale management required by fully NVFP4 $PV$.

Quantizing $P$ does not require a dynamically computed scale factor for each block, but should account for the limited range of FP8 `e4m3` and the numerical degradation caused by attention sinks. From Equation 4, each unnormalized probability generated by online softmax satisfies

$$0 < P_{ij}^{(t)} = e^{S_{ij}^{(t)} - m_i^{(t)}} \le 1.$$

MpFA therefore uses a fixed scale factor $s_P = 448$ for $P$:

$$\widehat{P}_{ij}^{(t)} = \mathrm{cast}_{\mathrm{e4m3}}\left(s_P P_{ij}^{(t)}\right) / s_P.$$

Because 448 is the largest normal `e4m3` value and $P_{ij}^{(t)} \le 1$, this analytic scale applies across layers, heads, and inputs without runtime `amax` statistics, per-block scale generation, or calibration. The remaining concern is the lower end of the FP8 range: in a forward traversal, an early attention sink can raise the running maximum and push ordinary-token probabilities into the `e4m3` subnormal range. MpFA therefore traverses $K$ and $V$ in reverse KV-block order to delay the sink block. Since online softmax is traversal-order invariant, reversal changes only the finite-precision rounding path, following FA3 [23].

### 4.2 Outlier Structure and Sequence Smoothing

NVFP4 block scaling assigns the dynamic range of each 16-element block according to its amax. Stable channel outliers within a block can therefore directly reduce the effective precision of 4-bit quantization. Figure 4 shows activations collected from a running service: one head from one layer of each model over the first 256 tokens. The magnitudes of $Q$ and $K$ form ridges along the channel dimension that persist across many tokens, while remaining approximately flat along the token dimension. Because NVFP4 blocks are partitioned along the channel axis, a small number of stable outlier channels can consume most of the representable range of their blocks. These outliers are determined by fixed model parameters rather than transient input tokens, making them suitable for smoothing. In contrast, $V$ varies comparably across the channel and token dimensions and is therefore not smoothed, consistent with prior work [29]. As $V$ is not centered, it requires no mean recovery and is unaffected by rank-one compensation. It uses FP8 `e4m3` (three mantissa bits versus one for NVFP4) with FP32 accumulation. Since $O = \sum_j P_j V_j$ is a convex combination, rounding errors in $V$ enter the output as a weighted average rather than a sum that grows linearly with sequence length.

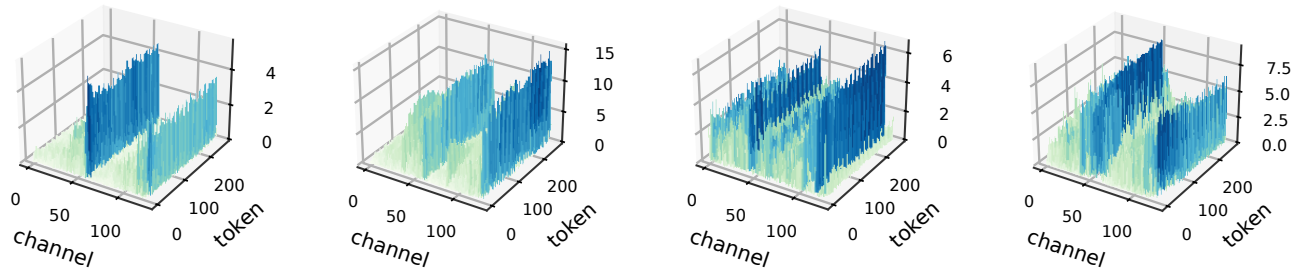
**Sequence centering.** To mitigate the stable channel outlier structure, we subtract a sequence-level mean for each layer and head before quantization. For the set of valid tokens $C$, we define

$$\bar{K}[h,:] = \frac{1}{|C|}\sum_{t\in C} K[t,h,:], \bar{Q}[h,:] = \frac{1}{|C|}\sum_{t\in C} Q[t,h,:] \quad (7)$$

The system obtains these two statistics during prefill and quantizes the centered values

$$Q_c = Q - \bar{Q}, \qquad K_c = K - \bar{K}.$$

We partition $Q_c$, $K_c$, and $V$ along the token dimension into $\{Q_{c,i}\}$, $\{K_{c,j}\}$, and $\{V_j\}$, with quantized blocks $\hat{Q}_{c,i}$, $\hat{K}_{c,j}$, and $\hat{V}_j$ and scales $s_{Q,i}$ and $s_{K,j}$. The removed means become recoverable compensation factors, while centering is fused into the quantization write path.




**(a)** Llama-3.1-8B $|Q|$ **(b)** Llama-3.1-8B $|K|$ **(c)** Qwen3-14B $|Q|$ **(d)** Qwen3-14B $|K|$

**Figure 4.** Distributions of $|Q|$ and $|K|$ for one head at layer 16 of Llama-3.1-8B and layer 20 of Qwen3-14B.

### 4.3 Rank-One Tensor Core Compensation

When both $Q$ and $K$ are centered, the removed mean terms must be restored before softmax. Expanding $S = QK^\top$ and defining a key-dependent compensation vector gives

$$S = Q_c K_c^\top + \mathbf{1}_M\big(\bar{Q}K_c^\top\big) + \big(Q\bar{K}^\top\big)\mathbf{1}_N^\top \tag{8}$$

$$\Delta_S[j] \triangleq \bar{Q} \cdot K_c[j,:]^\top \tag{9}$$

Here $\Delta_{S,j}$ denotes the entries of $\Delta_S[j]$ in KV block $j$; GQA uses one $\Delta_S$ vector per query head.

The third term is constant across keys within each softmax row and cancels because $\mathrm{softmax}(x + c\mathbf{1}) = \mathrm{softmax}(x)$. Only the key-dependent second term must be restored after centering:

$$\hat{S} = \mathrm{deq}\big(\hat{Q}_c\hat{K}_c^\top\big) + \mathbf{1}_M\Delta_S \tag{10}$$

Here $\Delta_S$ is added before masking and before updating the running maximum. This ordering prevents compensation from reaching masked positions and keeps the running maximum consistent with the normalized scores.

**Rank-one MMA implementation.** The compensation term in Equation 10 is the outer product $\mathbf{1}_M\Delta_S$ within a score tile. Rather than adding it element-wise after computing $\hat{Q}_c\hat{K}_c^\top$, we first issue a BF16 Tensor Core MMA instruction on the same score accumulator:

$$S_{\mathrm{acc}} \leftarrow \mathbf{1}_M\Delta_S, \qquad S_{\mathrm{acc}} \mathrel{+}= \mathrm{deq}\big(\hat{Q}_c\hat{K}_c^\top\big). \tag{11}$$

For the BF16 MMA reduction granularity $K$=16, the constant $A$ operand has 1 only in its first column, with the remaining columns and corresponding $\Delta_S$ components zeroed. The accumulation is therefore exactly $\mathbf{1}_M\Delta_S$, moving compensation from the softmax path to otherwise idle Tensor Cores without changing online softmax. Compared with element-wise addition, this introduces one BF16 rounding with relative error at most $2^{-8}$.

## 5 High Performance QK4PV8 FA Kernel

This section presents MpFA's unified QK4PV8 computation flow, followed by tensor memory reuse, rank-one compensation placement, and decode-specific optimizations.

### 5.1 Fine-Grained Pipeline

Algorithm 1 describes MpFA's two-stage execution flow. The first stage runs outside the attention operator: it computes $\bar{Q}$ and $\bar{K}$, centers $Q$ and $K$, quantizes $Q_c$ and $K_c$ to NVFP4, quantizes $V$ to FP8, and computes the per-token compensation vector $\Delta_S$ with one GEMV according to Equation 9. This work is fused with the inference framework's KV-cache write path (§6), keeping its overhead low. The second stage is the attention main loop, which consumes only the quantized data and compensation factors.

In the algorithm, $i$ and $j$ index query blocks and KV blocks, respectively. $\hat{Q}_{c,i}$ is the $i$-th quantized centered query block; $\hat{K}_{c,j}$ and $\hat{V}_j$ form the $j$-th KV block; and $s_{Q,i}$ and $s_{K,j}$ are their NVFP4 block scale factors. $S_{i,j}$ and $O_{i,j}$ denote the score and output states after processing KV block $j$, while $m_{i,j}$ and $\ell_{i,j}$ are the running maximum and normalization factor of online softmax. The only quantization inside the loop converts $P_{i,j}$ to FP8. Since its range has a known upper bound (§4.1), this conversion uses a fixed scale factor.

**Warp specialization.** MpFA retains the five warp roles of FA4 [27], but adapts them for QK4PV8: load, MMA, softmax, correction, and epilogue, as shown in Figure 5. The load warp issues asynchronous copies of $\hat{K}_{c,j}$, $s_{K,j}$, $\hat{V}_j$, the current query block and its scale factor, and $\Delta_{S,j}$ from global memory to on-chip memory. The MMA warp issues two MMAs for the current score state: a rank-one BF16 MMA first writes $\Delta_{S,j}$, followed by a block-scaled NVFP4 MMA that accumulates $\hat{Q}_{c,i}\hat{K}_{c,j}^\top$; an FP8 MMA then computes $\hat{P}_{i,j}\hat{V}_j$. The two softmax warpgroups apply the mask, update $m_{i,j}$ and $\ell_{i,j}$, and convert $P_{i,j}$ to $\hat{P}_{i,j}$ in place. The correction warpgroup rescales the output state and performs final normalization, while the epilogue warp writes the BF16 output back to global memory.

We double-buffer the query blocks ($Q_c$) to overlap MMA with softmax. One CTA processes two query stages, indexed by $r \in \{0, 1\}$. Each stage owns one $S^{(r)}$ and one $O^{(r)}$, corresponding to an $(S_{i,j}, O_{i,j})$ state in the algorithm. The probability matrix $\hat{P}_{i,j}$ receives no separate tensor-memory allocation; it is written into the upper half of the current $S^{(r)}$ region, and the FP8 $PV$ MMA reads it back from that region.

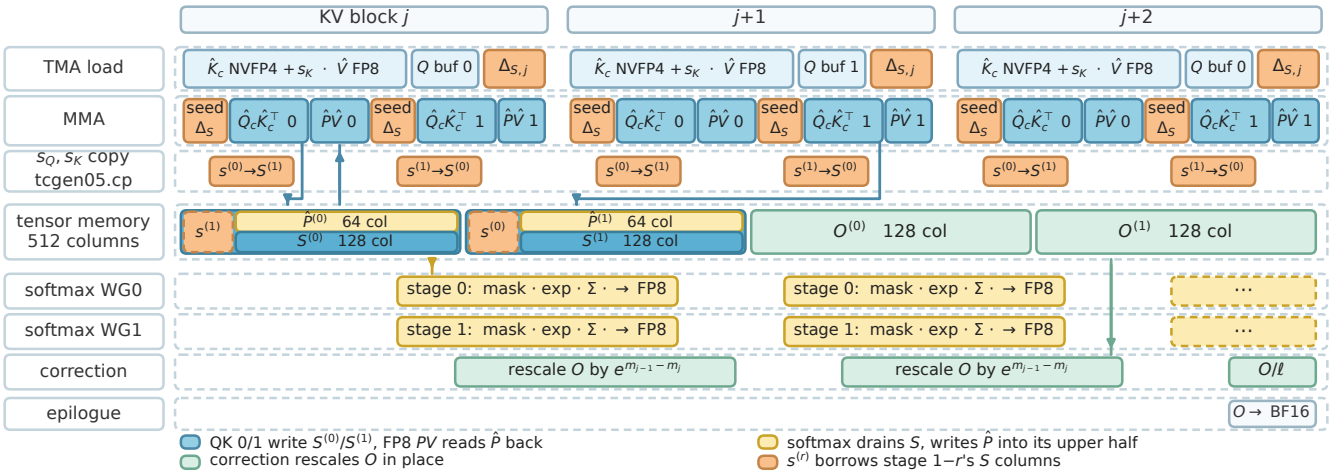



**Figure 5.** MpFA's steady-state pipeline. The seven execution channels for five warp roles within one CTA advance over consecutive KV blocks, together with their tensor memory accesses.

**Algorithm 1** Algorithm design of MpFA. $\Delta_{S,j}$ is the compensation vector. The $1/\sqrt{d}$ factor is folded into the $\lambda$.

**Input:** $Q, K, V$ in BF16; block sizes $B_q, B_{kv}$; $T_m = \lceil N_q/B_q \rceil$ query blocks and $T_n = \lceil N_{kv}/B_{kv} \rceil$ KV blocks; $M = B_q$
**Output:** $O$ in BF16

$\bar{Q} \leftarrow \mathrm{mean}(Q)$, $\bar{K} \leftarrow \mathrm{mean}(K)$ ▷ outside kernel
$Q_c \leftarrow Q - \bar{Q}$, $K_c \leftarrow K - \bar{K}$
$(s_Q, \hat{Q}_c) \leftarrow \mathrm{Quant}_{\mathrm{NVFP4}}(Q_c)$; $(s_K, \hat{K}_c) \leftarrow \mathrm{Quant}_{\mathrm{NVFP4}}(K_c)$
$\hat{V} \leftarrow \mathrm{Quant}_{\mathrm{FP8}}(V)$ ▷ fused into KV-cache write
$\Delta_S \leftarrow \mathrm{GEMV}_{\mathrm{bf16}}(\bar{Q}, K_c^\top)$ ▷ Equation 9
**for** $i = 1$ to $T_m$ **do** ▷ kernel main loop
  $m_{i,0} \leftarrow -\infty$, $\ell_{i,0} \leftarrow 0$, $O_{i,0} \leftarrow 0$
  **for** $j = 1$ to $T_n$ **do**
    load $\hat{Q}_{c,i}, s_{Q,i}, \hat{K}_{c,j}, s_{K,j}, \hat{V}_j, \Delta_{S,j}$ ▷ Equation 11
    $S_{i,j} \leftarrow \mathrm{MMA}_{\mathrm{bf16}}(\mathbf{1}_M, \Delta_{S,j}) + \mathrm{MMA}_{\mathrm{fp4}}(\hat{Q}_{c,i}, s_{Q,i}, \hat{K}_{c,j}, s_{K,j})$
    $m_{i,j} \leftarrow \max(m_{i,j-1}, \mathrm{rowmax}(S_{i,j}))$
    $P_{i,j} \leftarrow 2^{\lambda(S_{i,j} - m_{i,j})}$, $\lambda = \log_2 e/\sqrt{d}$ ▷ $1/\sqrt{d}$ folded here
    $\ell_{i,j} \leftarrow 2^{\lambda(m_{i,j-1} - m_{i,j})}\ell_{i,j-1} + \mathrm{rowsum}(P_{i,j})$ ▷ FP32 $P_{i,j}$
    $\hat{P}_{i,j} \leftarrow \mathrm{Quant}_{\mathrm{FP8}}(P_{i,j})$
    $O_{i,j} \leftarrow \mathrm{diag}\left(2^{\lambda(m_{i,j-1} - m_{i,j})}\right) O_{i,j-1} + \mathrm{MMA}_{\mathrm{fp8}}(\hat{P}_{i,j}, \hat{V}_j)$
  **end for**
  $O_i \leftarrow \mathrm{diag}(\ell_{i,T_n})^{-1} O_{i,T_n}$
**end for**
**return** $O = \{O_i\}_{i=1}^{T_m}$

Thus, stage $r$'s NVFP4 $QK^\top$ writes only $S^{(r)}$, its softmax reads $S^{(r)}$ and writes $\hat{P}^{(r)}$ in the upper half, and its FP8 $PV$ MMA reads $\hat{P}^{(r)}$ from the same region. The two stages alternate between $S^{(0)}/\hat{P}^{(0)}$ and $S^{(1)}/\hat{P}^{(1)}$, allowing one stage's softmax to overlap with the other stage's matrix multiplication. This double buffer hides softmax latency and enables reuse of the scale factors described next.

**Asynchronous pipeline for scale factors in tensor memory.** Each CTA is allocated 512 tensor memory columns. The score and output states of the two query stages require 128 columns each, so the four accumulators consume the entire capacity. NVFP4 scale factors for $Q$ and $K$ cannot receive independent allocations. MpFA resolves this conflict by exploiting asymmetric lifetimes: $S^{(r)}$ and $O^{(r)}$ must remain resident through a long softmax or main-loop interval, whereas $s_{Q,i}$ and $s_{K,j}$ are read only when the corresponding NVFP4 MMA is issued.

We asynchronously place FP8 scale factors in reused storage. The scale factors needed by stage $r$ are not placed on its own $S^{(r)}$ region; instead, they borrow columns from $S^{(1-r)}$: $s^{(0)}$ occupies the starting columns of $S^{(1)}$, and $s^{(1)}$ occupies those of $S^{(0)}$ (the dashed cells at the left of the score regions in Figure 5). The double-buffer timing makes this safe. While stage $r$ issues $QK^\top$, stage $1-r$'s score region is in the gap after softmax and $PV$ have consumed it but before the next MMA overwrites it. In contrast, $S^{(r)}$ is the target of the current MMA and cannot simultaneously hold its scale factors.

Scale-factor reads and writes form an independent pipeline with a phase distinct from data loading and matrix multiplication. Each query stage owns a set of `tcgen05.cp` copies and its own barrier phase (the two $s_Q, s_K$ copy boxes in Figure 5). Copies are issued early enough for the factors to arrive before the corresponding MMA; their transfer overlaps with the other stage's softmax and matrix multiplication and therefore does not enter the critical path. Correctness follows from two phase conditions: scale factors must arrive before their MMA is issued, and a new write to a score region can occur only after that region has been consumed by softmax and the $PV$ MMA. The probability matrix $P$ uses the same lifetime reuse but occupies only the upper half of the score region, requiring no additional tensor-memory allocation. This allows scale factors, attention scores, and outputs to remain resident within the fixed 512-column budget.

**Other attention optimizations.** MpFA builds the kernel on the CUTLASS and CuTe programming primitives. For causal attention, MpFA balances the triangular workload using longest-processing-time-first scheduling and overlaps TMA and MMA with a ping-pong KV pipeline. It also distributes softmax exponentiation between a polynomial approximation and hardware $2^x$ instructions to balance the FMA and SFU pipelines. Moreover, attention scaling is fused into exponentiation: $Q$ and $K$ remain in the unscaled inner-product domain, and the softmax constant $\lambda = \log_2 e/\sqrt{d}$ is folded into the exponent ($2^{\lambda(S-m)} = 2^{\lambda S-\lambda m}$), avoiding separate scaling of the scores and compensation term $\Delta_{S,j}$.

### 5.2 MMA Compensation

For each $(i, j)$, the MMA warp first issues $\mathrm{MMA}_{\mathrm{bf16}}(\mathbf{1}_M, \Delta_{S,j})$, then accumulates the NVFP4 $\mathrm{MMA}_{\mathrm{nvfp4}}(\hat{Q}_{c,i}, s_{Q,i}, \hat{K}_{c,j}, s_{K,j})$ into the same score state $S_{i,j}$. Because the compensation has already been written to the score accumulator before softmax, the softmax warp reads complete scores without an additional element-wise addition or a change to the online-softmax instruction sequence.

The compensation vector $\Delta_{S,j}$ is much smaller than $\hat{K}_{c,j}$ and $\hat{V}_j$. The loading warp prefetches it and publishes it with the same barrier phase as the current KV block. Rank-one MMA and NVFP4 $QK^\top$ consequently share one dependency and require no additional synchronization. Relative to the same build without smoothing, the prefill attention operator adds only 1.3–2.5%; placing compensation on the softmax warp costs 18.6–22.7% (§7.4).

### 5.3 Decode-Specific Optimizations

Decode uses the same representations and MMA precisions as prefill but is parallelism-bound. MpFA packs GQA heads, partitions the KV dimension, removes softmax work from padded rows, and merges the resulting local states.

GQA lets $H_q/H_{kv}$ query heads share one KV head, so MpFA packs each group into the MMA $M$ dimension and assigns one (request, KV-head) pair to each CTA. It partitions the history into fixed $8 \times 128$-token chunks; each CTA maintains local $(m, \ell, O)$ states in preallocated workspaces, making the path CUDA Graph capturable and increasing parallelism over split-KV-disabled BF16 FA4.

For GQA 32:8, FP4 MMA fixes $M = 128$ although each group has only four real rows. MpFA replicates these rows and processes their corresponding columns as four diagonal $32 \times 32$ tiles, reducing softmax work from $32 \times 128$ to $32 \times 32$ per row range and the full-tile $2^x$ evaluations from $128 \times 128$ to $128 \times 32$. The four warps produce four states per partition, yielding $4N_p$ states merged by log-sum-exp. Eight independent state streams and shared-memory scale prefetching hide the reduction's memory latency; the same strategy extends to other model shapes.

## 6 LLM Serving System

**Serving system.** To evaluate end-to-end performance, we integrate MpFA into SGLang [32] as an attention backend, following prior work [20]. The backend supports paged prefill and decode, GQA, masking, and CUDA Graph capture, while SGLang handles the remaining model execution. During decode, fixed-size cache pages and preallocated tensors keep the number of KV partitions constant after graph capture. MpFA returns BF16 outputs through the standard attention interface. With smoothing, the per-token compensation $\Delta_S$ follows the KV cache's physical page-table indices and resides in a side cache directly accessed during decode.

**Online quantization.** MpFA fuses NVFP4 quantization of $Q$, low-bit $K/V$ cache writes, and compensation generation into SGLang's cache-write path. It stores $K$ in NVFP4 with one `e4m3` scale byte per 16 elements and $V$ in FP8, allowing direct consumption without dequantization. For equal-dimensional $K$ and $V$, the layout uses 1/2.56 of the BF16 KV-cache footprint: 1/2 + 1/16 bytes per $K$ element and one byte per $V$ element, versus two bytes per element for each BF16 tensor. With smoothing, prefill computes per-layer, per-head $\bar{Q}$ and $\bar{K}$ over valid tokens and freezes them for the sequence lifetime. Decode centers only the newly appended token against these per-request means without recomputing them. This avoids degenerate single-token query centering that would shift the entire score correction to $\Delta_S$. For the new token, centering and compensation generation are fused into the cache-write kernel. Without smoothing, no side cache is allocated. Kernel measurements exclude quantization statistics and compensation generation; end-to-end measurements include quantization, sequence centering, and KV-cache writes.

## 7 Evaluation

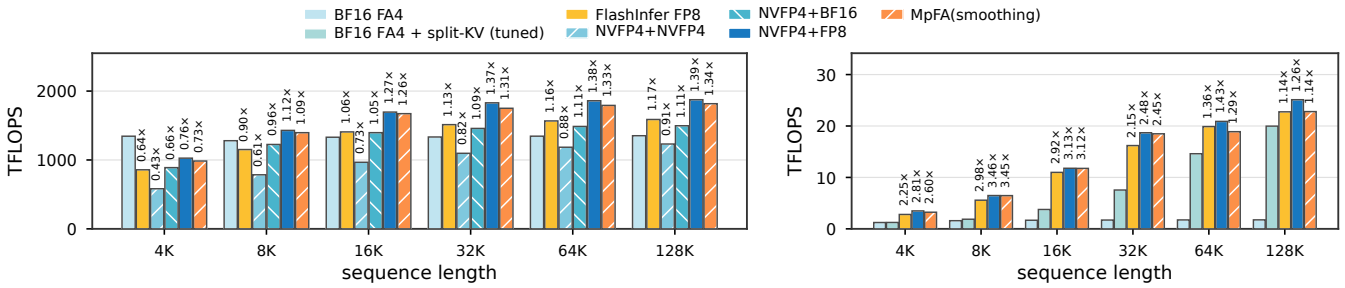
### 7.1 Experimental Setup

**Hardware and software.** We conduct all experiments on a single NVIDIA SM100 GPU using CUDA 13.0, PyTorch 2.11.0, and SGLang $v$0.5.15. For end-to-end experiments, we enable CUDA Graph capture and disable Radix Cache [32].

**Models and accuracy tasks.** We evaluate Llama-3.1-8B and Qwen3-14B, covering GQA 32:8 and 40:8; both have 128-dimensional heads. We extend Qwen3-14B from its native 40K to 128K using YaRN, while Llama-3.1-8B uses its native 128K context. This extension may affect accuracy but not performance measurements.

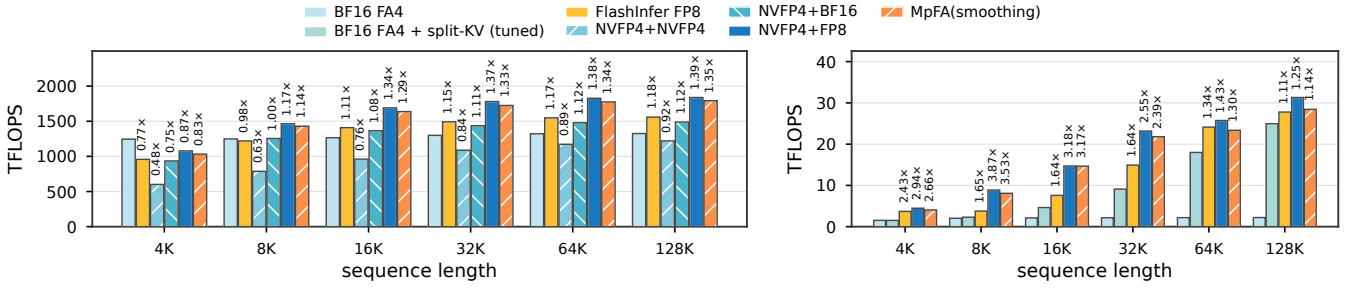
We evaluate MMLU [11] (5-shot, all 57 subjects), HellaSwag [28] (20-shot), WinoGrande [22] (0-shot), GSM8K [2] (5-shot with generated reasoning), and RULER-QA [12], which pads SQuAD and HotpotQA passages to 64K tokens. Each backend uses the same examples and prompts with greedy decoding ($temperature = 0$).

**Baselines.** We compare BF16 FA4, FP8 FlashInfer, and three precision assignments for $QK^\top$ and $PV$: NVFP4+NVFP4,




**(a)** causal prefill **(b)** decode

**Figure 6.** Attention-kernel throughput on an SM100 GPU with GQA 32:8 and *head_dim*=128, the shape of Llama-3.1-8B. Speedups above the bars use BF16 FA4 in (a) and *split-KV-enabled* BF16 FA4 in (b).




**(a)** causal prefill **(b)** decode

**Figure 7.** Attention-kernel throughput for Qwen3-14B (GQA 40:8). The baselines and speedup denominators match Figure 6.

NVFP4+BF16, and NVFP4+FP8. MpFA is the complete QK4V8 design with rank-one smoothing. For decode kernel tests, we tune BF16 FA4's split count per context length and use FlashInfer's default split-KV policy. MpFA instead uses fixed KV-partition and column-split configurations. For end-to-end tests, BF16 FA4 runs one CTA per request because its split-KV path preallocates workspaces per `num_splits` and cannot be captured by CUDA Graph. FlashInfer and MpFA use graph-compatible split-KV, with FlashInfer using an FP8 KV cache. Integrated as an SGLang attention backend, MpFA supports the same production features as the baselines: continuous batching, paged KV cache, CUDA Graph capture, and `head_dim` (128 for both models).

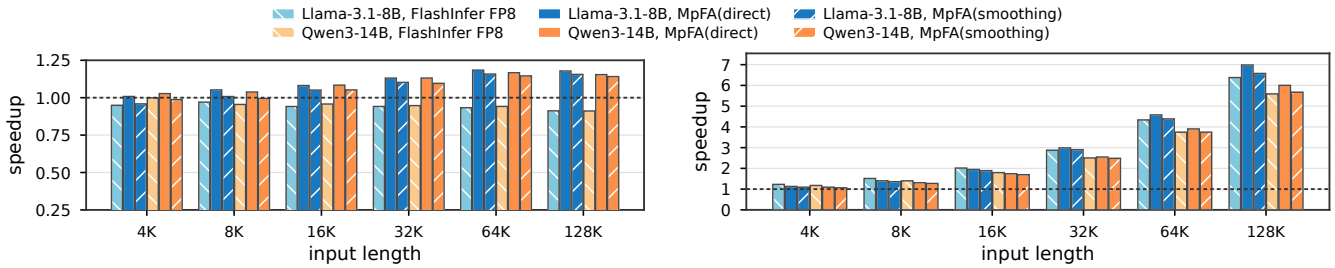
We measure prefill/decode kernel throughput, time to first token (TTFT), time per output token (TPOT), output-token throughput (OTPS) across concurrency levels, end-to-end accuracy, and ablations. Following prior work [5], we report the mean of five performance runs.

### 7.2 FlashAttention Kernel Performance

Figure 6 compares causal prefill and decode throughput across precision configurations, using BF16 FA4 and split-KV-enabled BF16 FA4 as their respective denominators. NVFP4 + FP8 is the hardware-efficient prefill assignment; decode also depends on KV partitioning and softmax scheduling. MpFA includes smoothing, while direct NVFP4 + FP8 isolates its compensation cost.

**Prefill workloads.** At 16K and above, measured throughput follows direct NVFP4 + FP8 > MpFA > NVFP4 + BF16 > BF16 > NVFP4 + NVFP4, consistent with §3.1. Across 16K–128K, MpFA sustains 1672–1817 TFLOPS and achieves a 1.31× geometric-mean speedup over BF16 FA4; the direct variant reaches 1.35×, showing that smoothing preserves most of the gain. FP8 FlashInfer reaches 1.12× over BF16, while MpFA is 1.17× faster than FlashInfer. The benefit follows from reducing the compute-intensive $QK^\top$ from FP8 to NVFP4. Qwen3-14B's GQA 40:8 shape shows the same trend (Figure 7), with 1.33×/1.15× speedups over BF16 FA4/FP8 FlashInfer.

Fixed overhead limits short sequences: at 4K, MpFA and its direct variant reach only 0.74× and 0.76× of BF16, versus 1.09× and 1.12× at 8K. The crossover therefore falls between 4K and 8K, below which matrix multiplication does not dominate. MpFA is thus suited to end-to-end inference at sequence lengths beyond 8K. NVFP4 + BF16 reaches 1.05–1.11× at ≥ 16K, confirming the benefit of quantizing $QK^\top$ alone. Relative to FP8, however, its $PV$ MMA and shared-memory traffic for $V$ both double, and it cannot use a low-bit $V$ cache. Fully NVFP4 is slowest from 4K onward, reaching only 0.87–0.90× BF16 at long sequences because of the additional $PV$ execution-path costs identified in §3.1.




**(a)** Time to first token (TTFT)

**(b)** Time per output token (TPOT)

**Figure 8.** End-to-end latency speedup over BF16 FA4 at concurrency 1.

**Decode workloads.** At 128K, MpFA and its direct variant reach only 22.8 and 25.1 TFLOPS, respectively, confirming that long-context decode is parallelism-bound rather than Tensor-Core-bound. We compare against three baselines with distinct deployment constraints.

The first is production-deployable BF16 FA4. Its split-KV path preallocates workspaces according to `num_splits` and cannot be captured by CUDA Graph, so SGLang uses one CTA to process the full history sequentially. Relative to this configuration, MpFA improves throughput from 2.63× at 4K to 13.04× at 128K; the direct variant reaches 2.83× and 14.37×, respectively.

The second is BF16 FA4 with split-KV enabled. We sweep `num_splits`$\in \{1, \ldots, 32\}$ and select the best value at each length; the optimum is 16 or 32 at $\geq$ 8K. Against this per-length-tuned baseline, MpFA achieves a 2.08× average speedup across 8K–128K without per-length tuning, compared with 2.17× for the direct variant. MpFA peaks at 3.45× at 8K and remains 1.14× faster at 128K, due to GQA packing, fixed KV partitions, column splitting, and parallel state merging. The speedup decreases with sequence length: partitioning exposes substantially more parallelism than the baseline at short-to-medium lengths, whereas longer KV histories make both systems HBM-bandwidth-bound, limiting the benefit of NVFP4 compute.

The third is FlashInfer's FP8 Blackwell FlashAttention, which supports split-KV under CUDA Graph. MpFA achieves a 1.06× geometric-mean speedup across 8K–128K, leading by 1.11–1.14× through 32K but falling to 0.95× at 64K and near parity (0.99×) at 128K. The direct variant reaches 1.11× in geometric mean and remains 1.05×/1.09× faster at 64K/128K. The gap reflects smoothing: the $\Delta_S$ side cache adds about 4% KV traffic, which is mostly absorbed by L2 through 32K but reaches HBM beyond 64K, making MpFA about 10% slower than the direct variant. The modest margin over FlashInfer also reflects the decode regime: at concurrency 1, reading the KV history dominates, so NVFP4's compute advantage over FP8 is largely unused. Moreover, block-scaled FP4 MMA requires $M$=128, while a GQA group has only $H_q/H_{kv}$ real query rows; row replication and column splitting cannot eliminate all padded work. These decode results generalize to

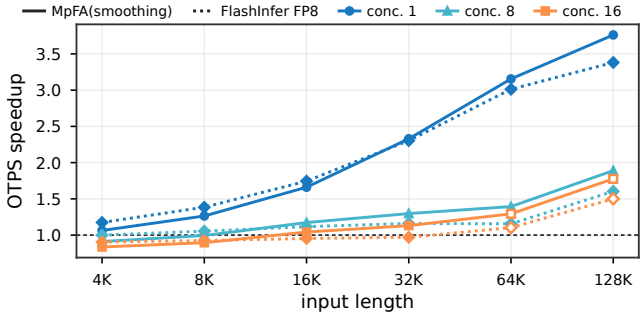



**Figure 9.** Output throughput of SGLang serving relative to BF16 FA4 for Qwen3-14B.

GQA 40:8 (Figure 7): MpFA peaks at 3.53× over split-KV BF16 FA4 at 8K and stays 1.14× at 128K. Its larger query-to-KV ratio feeds more compute per KV read, yielding 0.97×/1.02× of FP8 FlashInfer at 64K/128K.

### 7.3 End-to-End Inference Performance

**Latency.** Figure 8 compares MpFA, its direct variant, BF16 FA4, and FP8 FlashInfer under identical serving parameters. Across both models and 16K–128K, MpFA improves TTFT and TPOT over BF16 by 1.11× and 3.30× in geometric mean. At 128K, TPOT falls from 42.4 to 6.5 ms for Llama-3.1-8B (6.58×) and from 53.8 to 9.5 ms for Qwen3-14B (5.67×), slightly exceeding FlashInfer's 6.38×/5.59×.

Prefill gains are smaller because non-attention computation remains unchanged: at 128K, MpFA improves TTFT by 1.16×/1.14×. Although FP8 FlashInfer outperforms BF16 FA4 (Figure 6), separately producing its FP8 KV cache offsets this kernel gain end to end, giving 0.91–0.96× BF16 TTFT. MpFA fuses sequence centering, quantization, and cache writes (Figure 11), improving TTFT by up to 1.16× over BF16 FA4 and 1.16–1.27× over FP8 FlashInfer at 32K–128K.

Fixed overhead limits both latency metrics at short sequences, so we make no stable speedup claim for inputs of $\leq$ 2K. Relative to MpFA (direct), smoothing adds 1.5–7.3% to TTFT and 3.0–6.2% to TPOT over 4K–128K; reduced KV traffic dominates as context grows.

**Throughput.** Figure 9 reports output throughput relative to end-to-end BF16 FA4 with fixed input length and 1024

**Table 2.** End-to-end accuracy. Rec. is the fraction of NVFP4+FP8 loss (vs. BF16) that MpFA recovers.

| | Llama-3.1-8B | | | | | Qwen3-14B | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Dataset | BF16 | FP8 | NVFP4+FP8 | MpFA | Rec. | BF16 | FP8 | NVFP4+FP8 | MpFA | Rec. |
| MMLU | 0.685 | 0.683 | 0.644 (−0.041) | **0.674** (−0.011) | 73% | 0.787 | 0.787 | 0.776 (−0.011) | **0.785** (−0.002) | 82% |
| HellaSwag | 0.785 | 0.785 | 0.765 (−0.020) | **0.780** (−0.005) | 75% | 0.774 | 0.774 | 0.768 (−0.006) | **0.773** (−0.001) | 83% |
| WinoGrande | 0.752 | 0.751 | 0.735 (−0.017) | **0.745** (−0.007) | 59% | 0.727 | 0.726 | 0.712 (−0.015) | **0.720** (−0.007) | 53% |
| GSM8K | 0.761 | 0.757 | 0.653 (−0.108) | **0.736** (−0.025) | 77% | 0.887 | 0.882 | 0.877 (−0.010) | **0.882** (−0.005) | 50% |
| RULER-QA | 0.624 | 0.619 | 0.533 (−0.091) | **0.578** (−0.047) | 49% | 0.656 | 0.656 | 0.592 (−0.064) | **0.607** (−0.049) | 24% |

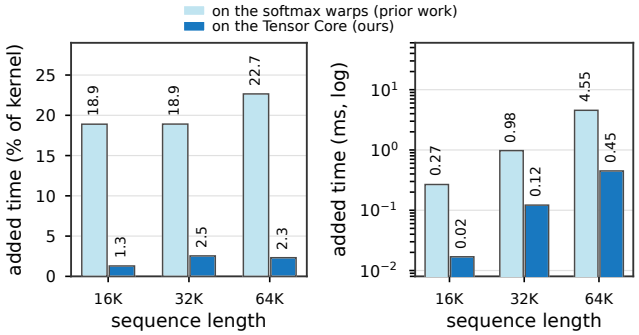
output tokens. At concurrency 1, MpFA achieves a 2.81× geometric-mean speedup across both models and 16K–128K, reaching 4.47×/3.76× at 128K (Llama-3.1-8B/Qwen3-14B). Absolute throughput then increases from 21.2 to 94.8 tok/s and from 16.2 to 61.0 tok/s. The smaller throughput gain than TPOT reflects the more modest 1.14–1.16× TTFT speedup. This speedup is measured against the production-deployable BF16 FA4, whose split-KV path cannot be graph-captured. However, the controlled kernel comparison with per-length split-KV tuning (Figure 6b) shows that MpFA remains faster at every length (2.08× on average and 1.14× at 128K), confirming that split-KV does not change the end-to-end performance ordering.

Relative to FP8 FlashInfer, MpFA leads across 32K–128K by a 1.06× geometric mean, reaching 1.11–1.12× at 128K on both models. The margin grows with context as sequence centering, quantization, and KV cache writes amortize and MpFA's lower KV cache traffic comes to dominate.

Higher concurrency narrows MpFA's advantage over BF16 but preserves the long-context benefit: at concurrency 8, it achieves a 1.33× average speedup across both models and 8K–128K, reaching 1.70×/1.89× at 128K. For Qwen3-14B at concurrency 16, it overtakes BF16 at 16K (1.04×) and reaches 1.13× at the fully resident 32K point. Batching supplies BF16 with parallelism, while the 2.56× KV-byte reduction persists.

The trend relative to FlashInfer holds under batching: at concurrency 8 the advantage is a 1.12× geometric mean across 8K–128K, from parity at 8K to 1.18–1.31× at 128K; for Qwen3-14B at concurrency 16 it is 1.09×/1.17× at the fully resident 16K/32K points. Points beyond 32K at concurrency 16 are capacity constrained because the KV pool admits only 11 and 5 requests at 64K and 128K, so we exclude them from aggregate results.

**Accuracy.** Table 2 compares four backends across five benchmarks and two models. FP8 FlashInfer stays within 0.006 of BF16 for every model–task pair. Direct NVFP4 + FP8 loses most on Llama-3.1-8B GSM8K (0.108) and RULER-QA (0.091); MpFA recovers 62.5% of the loss on average (24%–83% per pair). It recovers 49%–77% on Llama-3.1-8B, trailing FP8 by at most 0.009 on knowledge and commonsense tasks. Qwen3-14B has smaller direct losses (0.006–0.015 on single-token tasks), with 50%–83% recovery on the four shorter tasks and 24% on RULER-QA. Across both models the residual concentrates on RULER-QA and GSM8K, where RULER-QA recovery is 24%–49%: at 64K, error accumulated across many attention steps is less amenable to per-channel rank-one compensation. MpFA thus trades a few points on long-context and multi-step reasoning for a 2× lower-precision $QK^\top$ path that is faster than FP8 FlashInfer (§7.2), while restoring most of the accuracy loss without retraining.




**(a)** Relative overhead **(b)** Absolute overhead

**Figure 10.** Cost of the two compensation placements for causal prefill with the Llama-3.1-8B shape.

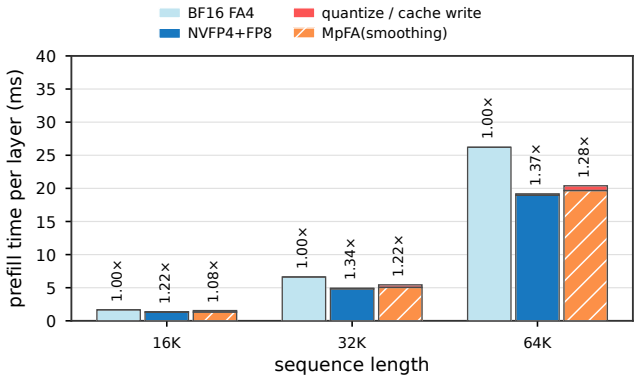



**Figure 11.** Per-layer prefill cost, including attention and the quantization / cache-write pass (Llama-3.1-8B).

## 7.4 Ablation Study

**Compensation placement.** Figure 10 compares two placements of the same compensation term $\Delta_S$ in QK4PV8 FA4.

Staging compensation in shared memory, synchronizing after each KV block, and adding it element-wise incurs 20.1% geometric-mean overhead across 16K–64K. With rank-one compensation, BF16 MMA first writes to tensor memory and NVFP4 $QK^\top$ accumulates into it, reducing overhead to 2.0%. At 64K, absolute overhead falls from 4.55 ms to 0.45 ms, or from more than half of the direct kernel's prefill advantage to less than one tenth. Both placements compute the same term: element-wise placement uses FP32 scalar addition, whereas rank-one placement carries $\Delta_S$ in BF16 and accumulates into FP32, adding one BF16 rounding with relative error no larger than $2^{-8}$. Table 2 includes this effect because MpFA uses rank-one compensation by default.

**Quantization overhead.** Quantization, sequence centering, and compensation generation remain part of end-to-end inference; MpFA fuses them with cache writes. Figure 11 includes this pass in the per-layer prefill cost and labels the resulting speedup over BF16 FA4. Across 32K and 64K, MpFA achieves a 1.25× geometric-mean speedup, compared with 1.34× for direct NVFP4 + FP8, preserving most of the kernel gain.

## 8 Related Work

FlashAttention [3, 4] reduces global-memory traffic through tiling and online softmax, and later work optimizes parallelism and data reuse for different GPUs. FlashAttention-3 (FA3) [23] adds warp specialization on SM90 (Hopper) GPUs to overlap matrix multiplication and softmax across warps, and FlashAttention-4 (FA4) [27] redesigns the pipeline for Blackwell tensor memory, fifth-generation Tensor Cores, and asymmetric hardware scaling, serving as our direct SM100 baseline. Other work adapts FlashAttention to the cache and data-reuse characteristics of Arm multicore CPUs [9], while a parallel line designs dedicated attention accelerators, such as the approximation- and pruning-based sparse-attention architectures SpAtten [24] and ELSA [10]. Collectively, these systems show that attention performance depends on co-designing algorithmic tiling with the hardware data path rather than optimizing solely for matrix multiplication peak throughput. MpFA retains FlashAttention's online softmax and FA4's SM100 execution model but addresses a different problem. While FA4 primarily uses BF16, MpFA moves $QK^\top$ and $PV$ to low-bit matrix multiplication paths and analyzes how block scaling, tensor memory capacity, and operand delivery jointly constrain the precision assignment.

Prior low-bit LLM quantization methods, including MR-GPTQ [7], AWQ [15], GPTQ [8], and SmoothQuant [25], target weights and activations in linear layers rather than attention activations. SageAttention [31] studies post-training quantization for attention: it smooths channel outliers in $K$, quantizes $QK^\top$ to INT8, and keeps the more error-sensitive $PV$ in FP16 with an FP16 accumulator. SageAttention2 [29] further quantizes $QK^\top$ to INT4, represents $P$ and $V$ in FP8, and applies more extensive outlier smoothing to recover accuracy. These systems share a post-training, plug-and-play approach and do not require both matrix multiplications to use the same precision. SageAttention3 [30] extends this approach to consumer Blackwell GPUs, using NVFP4 matrix multiplication for both $QK^\top$ and $PV$, together with $Q/K$ smoothing and probability quantization. HiFA4 [6] also provides training-free 4-bit attention, but targets the Ascend HIF4 NPU and its platform-specific formats and execution units. These systems do not address tensor memory, operand delivery, or asymmetric scaling between matmul and non-matmul units on data-center Blackwell GPUs. KV-cache quantization methods such as KIVI [16] lower cache precision to save memory and traffic, complementing quantized attention that consumes a low-bit cache without dequantization. At the serving level, schedulers exploiting the distinct prefill and decode profiles—through phase disaggregation [21] or prefill-decode overlap [14]—are orthogonal to MpFA, which accelerates the attention operator each phase relies on.

Different from prior work, MpFA is a high performance QK4PV8 attention kernel for data-center Blackwell GPUs. We characterize the SM100 data path and hardware behavior of low-bit attention, build mixed-precision pipelines specialized for prefill and decode, and integrate them into an end-to-end serving system. MpFA further applies rank-one Tensor Core MMA smoothing compensation, which recovers accuracy without placing additional compensation work on the already saturated softmax path.

## 9 Conclusion

We present MpFA, a training-free QK4PV8 FlashAttention kernel optimized for data-center Blackwell GPUs. Despite SM100's substantially higher 4-bit Tensor Core throughput, the overheads of online quantization, data movement, and tensor memory contention make fully NVFP4 attention inefficient. MpFA instead adopts mixed-precision NVFP4($QK^\top$) + FP8($PV$) and addresses its performance challenges through a fine-grained asynchronous pipeline, tensor memory reuse, and stage-specific optimizations for prefill and decode. To reduce quantization error without further stressing the softmax path, MpFA introduces rank-one Tensor Core MMA smoothing compensation. On SM100, this compensation recovers 62.5% of the accuracy loss with only 2.0% average kernel overhead. Across 16K–128K contexts, MpFA achieves average prefill speedups of 1.31–1.33× over BF16 FA4 and 1.15–1.17× over FP8 FlashInfer for GQA 32:8 and 40:8, and achieves 2.81× the end-to-end output throughput of BF16 FA4 across both evaluated models.

## References

[1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask

Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

[2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168* (2021).

[3] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Chencheng Deng, Weiling Yang, Jianbin Fang, and Dezun Dong. 2026. Demystifying ARM SME to Optimize General Matrix Multiplications. In *2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.

[6] Hui Dong, Yanzhao Li, Jie Gao, Chunlu Li, Zhiyuan Zhang, Yupeng Sun, Zhenyuan Chen, and Zhiqiang Zou. 2026. HiFA4: Training-Free 4-bit FlashAttention on Ascend HIF4 NPUs for LLM Inference. *arXiv:2607.04302* (2026).

[7] Vage Egiazarian, Roberto L. Castro, Denis Kuznedelev, Andrei Panferov, Eldar Kurtic, Shubhra Pandit, Alexandre Marques, Mark Kurtz, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2026. Bridging the Gap Between Promise and Performance for Microscaling FP4 Quantization. In *International Conference on Learning Representations (ICLR)*.

[8] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations (ICLR)*.

[9] Xiao Fu, Weiling Yang, Dezun Dong, and Xing Su. 2024. Optimizing Attention by Exploiting Data Reuse on ARM Multi-core CPUs. In *Proceedings of the 38th ACM International Conference on Supercomputing (ICS)*.

[10] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, Soosung Kim, Hyunji Choi, Sung Jun Jung, and Jae W. Lee. 2021. ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 692–705.

[11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations (ICLR)*.

[12] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models?. In *Conference on Language Modeling (COLM)*.

[13] Aaron Jarmusch and Sunita Chandrasekaran. 2026. Microbenchmarking NVIDIA's Blackwell Architecture: An In-depth Architectural Analysis. *arXiv:2512.02189* (2026).

[14] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[15] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*.

[16] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *International Conference on Machine Learning (ICML)*.

[17] NVIDIA. 2022. *NVIDIA H100 Tensor Core GPU Architecture*.

[18] NVIDIA. 2024. *CUDA C++ Programming Guide, Version 12.4*.

[19] NVIDIA. 2024. *NVIDIA Blackwell Architecture Technical Brief*. Technical Report. NVIDIA.

[20] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys)*.

[21] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Í nigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st Annual International Symposium on Computer Architecture (ISCA)*.

[22] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.

[23] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[24] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 97–110.

[25] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning (ICML)*.

[26] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems (MLSys)*.

[27] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. 2026. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. In *Proceedings of Machine Learning and Systems (MLSys)*.

[28] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.

[29] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. 2025. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-Thread INT4 Quantization. In *International Conference on Machine Learning (ICML)*.

[30] Jintao Zhang, Jia Wei, Pengle Zhang, Xiaoming Xu, Haofeng Huang, Haoxu Wang, Kai Jiang, Jun Zhu, and Jianfei Chen. 2025. SageAttention3: Microscaling FP4 Attention for Inference and an Exploration of 8-Bit Training. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[31] Jintao Zhang, Jia Wei, Pengle Zhang, Jun Zhu, and Jianfei Chen. 2025. SageAttention: Accurate 8-Bit Attention for Plug-and-Play Inference Acceleration. In *International Conference on Learning Representations (ICLR)*.

[32] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*.